Kế hoạch triển khai đồng bộ dự án FINMEM Swarm

Hiệp – GNN / Quant Lead
Tiến – LLM / Multi-Agent LLM Lead
Đạt – Database / Memory / Infrastructure Lead

Ngày 28 tháng 5 năm 2026

1 Mục tiêu tài liệu

Tài liệu này xác định kế hoạch triển khai đồng bộ cho dự án FINMEM Swarm. Mục tiêu chính là tránh tình trạng mỗi thành viên phát triển một phần riêng lẻ nhưng khi tích hợp lại không chạy được. Vì vậy, kế hoạch được xây dựng theo luồng dữ liệu chung của hệ thống, sau đó mới phân chia trách nhiệm cho từng thành viên. FINMEM Swarm được thiết kế như một hệ thống đa tác tử quản lý danh mục đầu tư. Hệ thống bao gồm các thành phần chính: Sector Agent, Text Stream, Graph Stream, Dynamic Memory Sharing, Debate Mechanism, Master Portfolio và Shadow Portfolio.

2 Nguyên tắc triển khai

Dự án không được chia theo kiểu mỗi người tự làm một thư mục riêng biệt. Thay vào đó, toàn bộ hệ thống phải được triển khai theo các contract dữ liệu chung.

2.1 Nguyên tắc chính

1. Mỗi module phải có input và output rõ ràng.
2. Mọi giao tiếp giữa các thành viên phải thông qua schema chung.
3. Không truyền object nội bộ trực tiếp giữa các module.
4. Mỗi artifact phải có producer, consumer và nơi lưu trữ.
5. Mỗi milestone phải có integration demo chạy được.
6. Không dùng confidence tự khai của LLM làm tín hiệu định lượng chính.
7. Text Stream sử dụng pretrained text encoder để tạo vector h^{text}.
8. Shadow Portfolio dùng để đánh giá hiệu suất giả định và cập nhật trạng thái tâm lý của Sector Agent.
9. Master Portfolio là nơi duy nhất quản lý vốn thật và thực thi lệnh cuối.

3 Kiến trúc tổng thể

Luồng xử lý tổng quát của hệ thống:

$$\begin{aligned}\text{Raw Data} &\to \text{Data Engine} \to \text{Sector Agents} \\ &\to \text{Agent Output} \to \text{Quant Layer} \to \text{Graph/GNN} \\ &\to \text{Fusion} \to \text{Risk Policy} \to \text{Allocator} \\ &\to \text{Master Portfolio} \to \text{Evaluation}\end{aligned}$$

Trong đó:

- Đạt chịu trách nhiệm cung cấp dữ liệu, memory store, event store và runtime.
- Tiến chịu trách nhiệm xây dựng Sector Agent, context, LLM reasoning, text embedding, memory sharing và debate.
- Hiệp chịu trách nhiệm chuẩn hóa signal, tính return, xây graph, GNN, fusion, risk policy, allocation và benchmark.

4 Luồng toán học cốt lõi

4.1 Dữ liệu giá và lợi suất

Giả sử hệ thống có N Sector Agent tương ứng với N tài sản:

$$A = \{A_1, A_2, \ldots, A_N\}$$

Vector giá tại ngày t:

$$P_t = [P_{1,t}, P_{2,t}, \ldots, P_{N,t}]$$

Lợi suất của tài sản i từ ngày t sang ngày $t+1$:

$$r_{i,t+1} = \log\left(\frac{P_{i,t+1}}{P_{i,t}}\right)$$

Vector lợi suất toàn bộ tài sản:

$$r_{t+1} = [r_{1,t+1}, r_{2,t+1}, \ldots, r_{N,t+1}]$$

4.2 Memory của Sector Agent

Mỗi Sector Agent có bộ nhớ phân tầng:

$$M_i = \{M_i^{shallow}, M_i^{intermediate}, M_i^{deep}\}$$

Trong đó:

- $M_i^{shallow}$ lưu tin tức ngắn hạn.
- $M_i^{intermediate}$ lưu báo cáo quý hoặc thông tin trung hạn.
- M_i^{deep} lưu báo cáo năm, tri thức nền tảng và reflection.

Context văn bản của agent i tại ngày t:

$$context_{i,t} = [M_{i,t}^{shallow}, M_{i,t}^{intermediate}, M_{i,t}^{deep}, p_{i,t}^{S}, \psi_{i,t}]$$

Trong đó $p_{i,t}^{S}$ là trạng thái hiệu suất của Shadow Portfolio, còn $\psi_{i,t}$ là trạng thái tâm lý của agent.

4.3 Text Stream

Text Stream dùng pretrained text encoder để biến context thành vector:

$$h_{i,t}^{text} = TextEncoder(context_{i,t})$$

Gom toàn bộ agent:

$$H_t^{text} = \begin{bmatrix} h_{1,t}^{text} \\ h_{2,t}^{text} \\ \vdots \\ h_{N,t}^{text} \end{bmatrix} \in \mathbb{R}^{N \times d_{text}}$$

Text score được sinh bởi một prediction head có thể train:

$$\hat{y}_t^{text} = \sigma(H_t^{text} W_p + b_p)$$

Trong đó $\hat{y}_{i,t}^{text}$ biểu diễn xu hướng tích cực của tài sản i theo dữ liệu văn bản.

4.4 Local decision của Sector Agent

Sector Agent sinh quyết định cục bộ:

$$d_{i,t} \in \{buy, hold, sell\}$$

Quy đổi thành action signal:

$$a_{i,t} = \begin{cases} +1, & \text{nếu } d_{i,t} = buy \\ 0, & \text{nếu } d_{i,t} = hold \\ -1, & \text{nếu } d_{i,t} = sell \end{cases}$$

Vector action:

$$a_t = [a_{1,t}, a_{2,t}, \ldots, a_{N,t}]$$

Action signal chỉ là tín hiệu cục bộ của Sector Agent, không phải lệnh giao dịch thật.

4.5 Shadow Portfolio

Mỗi Sector Agent có một Shadow Portfolio riêng. Shadow Portfolio không nắm giữ tiền thật, chỉ mô phỏng hiệu suất giả định nếu giao dịch theo tín hiệu của agent đó.
Lợi suất giả định của agent i:

$$R^S_{i,t+1} = a_{i,t} \cdot r_{i,t+1}$$

Vector lợi suất Shadow Portfolio:

$$R^S_{t+1} = a_t \odot r_{t+1}$$

Cập nhật giá trị Shadow Portfolio:

$$V^S_{i,t+1} = V^S_{i,t}(1 + R^S_{i,t+1})$$

ROI cá nhân của agent i:

$$ROI^S_{i,t} = \frac{V^S_{i,t}}{V^S_{i,0}} - 1$$

Trạng thái hiệu suất giả định:

$$p^S_{i,t} = [ROI^S_{i,t}, rolling_return^S_{i,t}, sharpe^S_{i,t}, drawdown^S_{i,t}, win_rate^S_{i,t}]$$

Trạng thái tâm lý:

$$\psi_{i,t} = PsychologyUpdate(p^S_{i,t})$$

Ví dụ:

$$\psi_{i,t} = \begin{cases} Greed, & \text{nếu ROI cao, drawdown thấp} \\ Fear, & \text{nếu ROI âm, drawdown cao} \\ Neutral, & \text{trường hợp còn lại} \end{cases}$$

4.6 Graph Stream

Graph giữa các Sector Agent được biểu diễn bởi:

$$G_t = (V, E)$$

Ma trận kề:

$$A_t \in \mathbb{R}^{N \times N}$$

Node feature đầu vào cho GNN:

$$H_t^0 = concat(market_features_t, a_t, \hat{y}_t^{text}, P_t^S, \Psi_t)$$

Trong đó P_t^S là ma trận shadow features và Ψ_t là biểu diễn trạng thái tâm lý.

4.7 Graph Attention Network

GAT tính attention từ agent j sang agent i:

$$\alpha_{ij} = softmax_j\left(LeakyReLU\left(a^T[W_g h_i \| W_g h_j]\right)\right)$$

Biểu diễn graph của agent i:

$$h_{i,t}^{graph} = ReLU\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} W_g h_{j,t}^0\right)$$

Gom toàn bộ agent:

$$H_t^{graph} = GAT(H_t^0, A_t)$$

Ma trận attention:

$$\mathcal{A}_t = \begin{bmatrix} \alpha_{11} & \alpha_{12} & \cdots & \alpha_{1N} \\ \alpha_{21} & \alpha_{22} & \cdots & \alpha_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ \alpha_{N1} & \alpha_{N2} & \cdots & \alpha_{NN} \end{bmatrix}$$

4.8 Dynamic Memory Sharing

Memory từ agent A được chia sẻ sang agent B nếu:

$$\alpha_{BA} \cdot I(m_A) > \theta$$

Trong đó:

- α_{BA} là mức ảnh hưởng từ agent A sang agent B.
- $I(m_A)$ là điểm quan trọng của memory từ agent A.
- θ là ngưỡng kích hoạt chia sẻ memory.

Khi điều kiện đúng:

$$W_B \leftarrow W_B \cup \{m_A, source = A, reason = \alpha_{BA}\}$$

4.9 Gated Fusion

Nếu H_t^{text} và H_t^{graph} chưa cùng số chiều, cần chiếu về cùng không gian:

$$\tilde{H}_t^{text} = H_t^{text} W_{text}$$

$$\tilde{H}_t^{graph} = H_t^{graph} W_{graph}$$

Gate:

$$Z_t = \sigma\left([\tilde{H}_t^{text} \| \tilde{H}_t^{graph}] W_z + b_z\right)$$

Fusion:

$$H_t^{final} = Z_t \odot \tilde{H}_t^{text} + (1 - Z_t) \odot \tilde{H}_t^{graph}$$

Decision score:

$$s_t = \sigma(H_t^{final} W_o + b_o)$$

Trong đó:

$$s_t = [s_{1,t}, s_{2,t}, \ldots, s_{N,t}]$$

$s_{i,t}$ gần 1 nghĩa là ưu tiên tăng phân bổ cho tài sản i, còn $s_{i,t}$ gần 0 nghĩa là ưu tiên giảm phân bổ.

4.10 Conflict Detection

Graph score:

$$\hat{y}_t^{graph} = \sigma(H_t^{graph} W_{gout} + b_{gout})$$

Conflict flag:

$$C_{i,t} = \begin{cases} 1, & \text{nếu } sign(\hat{y}_{i,t}^{text} - 0.5) \neq sign(\hat{y}_{i,t}^{graph} - 0.5) \\ 0, & \text{ngược lại} \end{cases}$$

4.11 Filter-then-Debate

Decision mode của tài sản i:

$$DecisionMode_{i,t} = \begin{cases} fastExecution, & \text{nếu } s_{i,t} > 0.8 \text{ hoặc } s_{i,t} < 0.2 \\ Debate, & \text{nếu } 0.3 \leq s_{i,t} \leq 0.7 \text{ hoặc } C_{i,t} = 1 \\ Review, & \text{trường hợp còn lại} \end{cases}$$

4.12 Master Portfolio

Allocator quyết định vector tỷ trọng vốn thật:

$$w_t = [w_{1,t}, w_{2,t}, \ldots, w_{N,t}]$$

Ràng buộc cơ bản:

$$\sum_{i=1}^{N} w_{i,t} \leq 1$$

$$0 \leq w_{i,t} \leq w_{max}$$

Lợi suất Master Portfolio:

$$R_{t+1}^{M} = w_t^T r_{t+1}$$

Nếu có transaction cost:

$$R_{t+1}^{M} = w_t^T r_{t+1} - cost_t$$

Cập nhật giá trị Master Portfolio:

$$V_{t+1}^{M} = V_t^{M}(1 + R_{t+1}^{M})$$

5 Cấu trúc thư mục và file cần bổ sung

Repo FINMEM gốc hiện có cấu trúc lõi gồm `config`, `data`, `puppy`, `run.py` và các script chạy mẫu. Để mở rộng sang FINMEM Swarm, cần giữ lại lõi FINMEM gốc để làm baseline và bổ sung các thư mục mới cho data engine, memory infrastructure, swarm agents, graph, fusion, portfolio, evaluation, API và experiment.

5.1 Nguyên tắc mở rộng thư mục

1. Không xóa hoặc sửa mạnh lõi FINMEM gốc ở giai đoạn đầu.
2. Các phần mở rộng của FINMEM Swarm phải được đặt trong module mới.
3. Các module giao tiếp với nhau thông qua artifact schema chung.
4. Mỗi thư mục mới phải có owner chính và consumer rõ ràng.
5. Mọi kết quả trung gian phải có thể lưu lại để phục vụ debugging, benchmark và ablation.

5.2 Cấu trúc thư mục đề xuất

```
FinMem/
|-- config/
|   |-- config.toml
|   `-- swarm/
|       |-- swarm_tsla_nvda_aapl.yaml
|       |-- data_config.yaml
|       |-- model_config.yaml
|       `-- experiment_config.yaml
|
|-- data/
|   |-- raw/
|   |-- parquet/
|   |-- duckdb/
|   |-- sqlite/
|   |-- faiss/
|   |-- event_store/
|   |-- checkpoints/
|   |-- model_artifacts/
|   `-- reports/
|
|-- puppy/
|   |-- agent.py
|   |-- memorydb.py
|   |-- environment.py
|   |-- portfolio.py
|   |-- reflection.py
|   |-- prompts.py
|   |
|   |-- common/
|   |   |-- schemas.py
|   |   |-- enums.py
|   |   |-- types.py
|   |   `-- time_utils.py
|   |
|   |-- data_engine/
|   |   |-- schemas.py
|   |   |-- parquet_loader.py
|   |   |-- duckdb_store.py
|   |   |-- multi_asset_env.py
|   |   |-- event_store.py
|   |   `-- result_store.py
|   |
|   |-- memory_ext/
|   |   |-- vector_store.py
|   |   |-- faiss_store.py
|   |   |-- sqlite_memory_store.py
|   |   |-- memory_namespace.py
|   |   `-- memory_sharing.py
```

```
|   |
|   |-- swarm/
|   |   |-- protocols.py
|   |   |-- sector_agent.py
|   |   |-- allocator_agent.py
|   |   |-- orchestrator.py
|   |   |-- run_swarm.py
|   |   |-- context_builder.py
|   |   `-- psychology.py
|   |
|   |-- fusion/
|   |   |-- text_encoder.py
|   |   |-- text_score_head.py
|   |   |-- gated_fusion.py
|   |   |-- dataset.py
|   |   |-- loss.py
|   |   |-- train_loop.py
|   |   `-- model_io.py
|   |
|   |-- graph/
|   |   |-- graph_builder.py
|   |   |-- correlation_graph.py
|   |   |-- supply_chain_graph.py
|   |   |-- graph_features.py
|   |   |-- risk_router.py
|   |   |-- gat_model.py
|   |   |-- graph_dataset.py
|   |   `-- graph_train_loop.py
|   |
|   |-- quant/
|   |   |-- signal_utils.py
|   |   |-- return_utils.py
|   |   |-- risk_rules.py
|   |   |-- shadow_metrics.py
|   |   `-- agent_reliability.py
|   |
|   |-- portfolio_ext/
|   |   |-- shadow_portfolio.py
|   |   |-- master_portfolio.py
|   |   |-- execution_policy.py
|   |   `-- allocation_engine.py
|   |
|   |-- debate/
|   |   |-- prompts.py
|   |   |-- conflict_detector.py
|   |   |-- debate_loop.py
|   |   `-- judge.py
|   |
|   |-- evaluation/
|   |   |-- metrics.py
```

```
|   |   |-- backtest_report.py
|   |   |-- plot_metrics.py
|   |   |-- explainability_report.py
|   |   `-- ablation_runner.py
|   |
|   |-- api/
|   |   |-- main.py
|   |   |-- routes_agents.py
|   |   |-- routes_graph.py
|   |   |-- routes_memory.py
|   |   |-- routes_debate.py
|   |   `-- routes_portfolio.py
|   |
|   `-- messaging/
|       |-- events.py
|       |-- broker.py
|       `-- redis_bus.py
|
|-- scripts/
|   |-- build_duckdb.py
|   |-- run_milestone_1_mock.py
|   |-- run_milestone_2_agents.py
|   |-- run_milestone_3_shadow.py
|   |-- run_milestone_4_graph.py
|   |-- run_milestone_5_fusion.py
|   |-- run_milestone_6_portfolio.py
|   |-- run_milestone_7_debate.py
|   |-- run_ablation.py
|   `-- run_batch_experiments.py
|
|-- docs/
|   |-- system_contract.md
|   |-- data_flow.md
|   |-- experiment_protocol.md
|   |-- module_ownership.md
|   |-- quant_design.md
|   |-- llm_agent_design.md
|   |-- database_memory_design.md
|   |-- debate_examples.md
|   |-- memory_sharing_examples.md
|   `-- experiment_results.md
|
|-- run.py
|-- docker-compose.yml
`-- Dockerfile
```

5.3 Ý nghĩa các nhóm thư mục mới

Thư mục	Owner chính	Vai trò
`puppy/common/`	Cả nhóm	Định nghĩa schema, enum, type dùng chung để tránh mỗi người tự định nghĩa một kiểu dữ liệu riêng.
`puppy/data_engine/`	Đạt	Đọc dữ liệu nhiều symbol, thay pickle bằng DuckDB/Parquet, tạo MultiAssetEnvironment và EventStore.
`puppy/memory_ext/`	Đạt, Tiến	Tách FAISS/SQLite khỏi logic agent; Tiến dùng API này để retrieve và inject memory.
`puppy/swarm/`	Tiến, Đạt	Bọc LLMAgent thành SectorAgent, tạo Orchestrator và chạy daily swarm loop.
`puppy/fusion/`	Tiến, Hiệp	Tiến tạo `h_text`; Hiệp train text score head và gated fusion.
`puppy/graph/`	Hiệp	Xây graph, GAT, risk router và memory share routes.
`puppy/quant/`	Hiệp	Tính signal, return, label, risk rules và shadow metrics.
`puppy/portfolio_ext/`	Hiệp, Đạt	Quản lý Shadow Portfolio, Master Portfolio, allocation và order execution simulation.
`puppy/debate/`	Tiến, Hiệp	Hiệp phát hiện conflict; Tiến xử lý debate và judge.
`puppy/evaluation/`	Hiệp	Metrics, report, plot, ablation và explainability report.
`puppy/api/`	Đạt	API quan sát agent, graph, memory sharing, debate và portfolio.
`puppy/messaging/`	Đạt	Event bus nội bộ hoặc Redis Pub/Sub cho các sự kiện memory sharing, debate và graph risk.

6 Luồng triển khai sau khi có các thư mục mới

6.1 Luồng dữ liệu cấp hệ thống

Sau khi triển khai, toàn bộ hệ thống phải đi theo chuỗi artifact sau:

$$MarketStep_t \rightarrow AgentOutput_t \rightarrow QuantFeatureTable_t \rightarrow ShadowPortfolioState_t \rightarrow GraphOutput_t \rightarrow FusionOutput_t \rightarrow DebateVerdict_t \rightarrow AllocationDecision_t \rightarrow PortfolioSnapshot_t$$

Trong đó không phải artifact nào cũng xuất hiện ở mọi nhánh. Ví dụ, `DebateVerdict` chỉ xuất hiện khi `FusionOutput` báo trạng thái grey zone hoặc conflict.

6.2 Bảng luồng artifact

Artifact	File sinh ra	File sử dụng	Ý nghĩa trong luồng
`MarketStep`	`data_engine/multi_asset_env.py`	`swarm/sector_agent.py`, `quant/return_utils.py`	Gói dữ liệu ngày t gồm price, news, filings cho nhiều symbol.
`MemoryQueryResult`	`memory_ext/faiss_store.py`, `memory_ext/sqlite_memory_store.py`	`swarm/context_builder.py`	Kết quả retrieve memory từ shallow, intermediate, deep hoặc reflection memory.
`AgentContext`	`swarm/context_builder.py`	`swarm/sector_agent.py`, `fusion/text_encoder.py`	Context gồm memory, shared context, shadow state và psychology state.
`AgentOutput`	`swarm/sector_agent.py`	`quant/signal_utils.py`, `graph/graph_features.py`, `fusion/text_score_head.py`	Output của SectorAgent gồm decision, action signal, reason, memory ids và h^{text}.
`QuantFeatureTable`	`quant/signal_utils.py`, `quant/return_utils.py`	`graph/graph_features.py`, `portfolio_ext/shadow_portfolio.py`	Chứa action signal, return, label và feature định lượng.
`ShadowPortfolioState`	`portfolio_ext/shadow_portfolio.py`, `quant/shadow_metrics.py`	`swarm/context_builder.py`, `swarm/psychology.py`	Trạng thái giả định của từng SectorAgent, dùng để cập nhật psychology state.
`GraphOutput`	`graph/gat_model.py`, `graph/risk_router.py`	`fusion/gated_fusion.py`, `memory_ext/memory_sharing.py`	Chứa H^{graph}, attention matrix, graph risk score và memory share routes.
`MemoryShareEvent`	`graph/risk_router.py`, `memory_ext/memory_sharing.py`	`swarm/context_builder.py`, `memory_ext/sqlite_memory_store.py`	Ghi lại agent nào chia sẻ memory sang agent nào và vì sao.

`FusionOutput`	`fusion/gated_fusion.py`	`quant/risk_rules.py`, `debate/conflict_detector.py`	Chứa fusion score, gate text, gate graph, conflict flag và decision mode.
`DebateRequest`	`debate/conflict_detector.py`	`debate/debate_loop.py`	Yêu cầu debate khi fusion score vùng xám hoặc text/graph conflict.
`DebateVerdict`	`debate/judge.py`	`portfolio_ext/allocation_engine.py`	Kết luận debate, ví dụ keep, reduce, override hold hoặc override sell.
`AllocationDecision`	`portfolio_ext/allocation_engine.py`, `portfolio_ext/execution_policy.py`	`portfolio_ext/master_portfolio.py`	Quyết định tỷ trọng vốn thật và order delta cho Master Portfolio.
`PortfolioSnapshot`	`portfolio_ext/master_portfolio.py`	`evaluation/metrics.py`, `evaluation/backtest_report.py`	Trạng thái danh mục thật gồm cash, position, equity curve và drawdown.

7 Sơ đồ luồng chạy dự kiến theo file và hàm

Phần này mô tả luồng chạy dự kiến của toàn bộ hệ thống FINMEM Swarm ở mức file và hàm. Mục tiêu là làm rõ script nào đóng vai trò entry point, module nào được gọi tiếp theo, artifact nào được sinh ra và artifact đó được module nào sử dụng.

7.1 Entry point của hệ thống

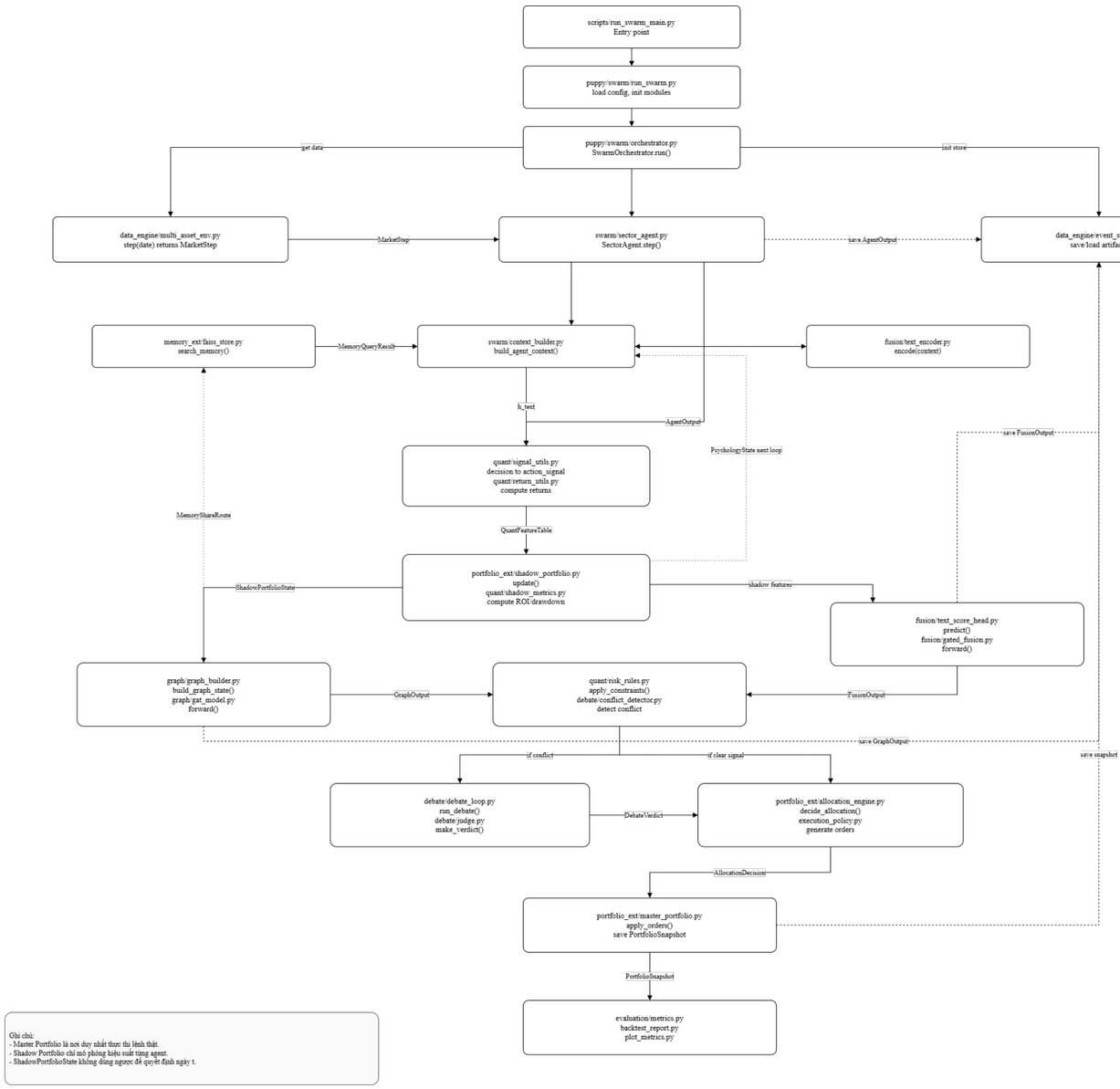
FINMEM gốc vẫn giữ `run.py` để chạy baseline single-agent. FINMEM Swarm bổ sung entry point mới:

`scripts/run_swarm_main.py`
↓
`puppy/swarm/run_swarm.py`
↓
`puppy/swarm/orchestrator.py`

Trong đó:

- `run.py`: giữ lại để chạy FINMEM gốc làm baseline.
- `scripts/run_swarm_main.py`: script chạy hệ thống FINMEM Swarm.
- `puppy/swarm/run_swarm.py`: đọc config, khởi tạo store, environment, agent và model.
- `puppy/swarm/orchestrator.py`: điều phối toàn bộ daily loop.

7.2 Sơ đồ runtime tổng thể



Hình 1: Luồng chạy dự kiến của FINMEM Swarm theo file và hàm chính.

7.3 Bảng ánh xạ file, hàm và artifact

File	Hàm/class chính	Artifact sinh ra	Module sử dụng tiếp theo
`scripts/run_swarm_main.py`	`main()`	Runtime command	Gọi `puppy/swarm/run_swarm.py`.

puppy/swarm/run_swarm.py	load_config(), build_modules(), run()	Config object, initialized modules	Truyền vào SwarmOrchestrator.
puppy/swarm/orchestrator.py	SwarmOrchestrator.run(), run_day(date)	Daily execution loop	Gọi environment, agents, quant, graph, fusion, allocation.
puppy/data_engine/multi_asset_env.py	step(date), get_market_step(date)	MarketStep	SectorAgent.step() và Quant Layer.
puppy/memory_ext/faiss_store.py	search_memory(), add_memory()	MemoryQueryResult	context_builder.build_agent_context().
puppy/memory_ext/sqlite_memory_store.py	get_memory_metadata(), save_memory_metadata()	Memory metadata	Context builder và memory sharing.
puppy/swarm/context_builder.py	build_agent_context()	AgentContext	SectorAgent.step() và text_encoder.encode().
puppy/swarm/sector_agent.py	SectorAgent.step()	AgentOutput	EventStore, Quant Layer, Fusion/Text Stream.
puppy/fusion/text_encoder.py	encode(context)	h_text	Text score head và Gated Fusion.
puppy/quant/signal_utils.py	validate_signal(), decision_to_action_signal()	action_signal	Return utils, graph features, shadow portfolio.
puppy/quant/return_utils.py	compute_log_return(), build_labels()	returns, labels	Graph features, metrics, shadow portfolio.
puppy/portfolio_ext/shadow_portfolio.py	update_shadow_portfolio()	ShadowPortfolioState	Psychology update và graph node features.
puppy/quant/shadow_metrics.py	compute_shadow_metrics()	ROI, drawdown, win rate, Sharpe	swarm/psychology.py, graph features.
puppy/swarm/psychology.py	update_psychology_state()	PsychologyState	Context builder ở vòng giao dịch sau.
puppy/graph/graph_features.py	build_node_features()	H_t^0	Graph builder và GAT.
puppy/graph/graph_builder.py	build_adjacency_matrix(), build_graph_state()	A_t, graph state	GAT model.
puppy/graph/gat_model.py	forward(H_t^0,A_t)	H_graph, Alpha_t	Fusion và risk router.
puppy/graph/risk_router.py	build_memory_share_routes()	MemoryShareRoute	Memory sharing module và SectorAgent context.
puppy/memory_ext/memory_sharing.py	apply_memory_share_routes()	MemoryShareEvent	EventStore và target SectorAgent.
puppy/fusion/text_score_head.py	predict_text_score()	text_score	Gated Fusion và conflict detector.
puppy/fusion/gated_fusion.py	forward(H_text,H_graph)	FusionOutput	Risk rules và conflict detector.
puppy/debate/conflict_detector.py	detect_conflict(), build_debate_request()	DebateRequest	Debate loop.
puppy/debate/debate_loop.py	run_debate()	Debate transcript	Judge.

puppy/debate/judge.py	make_verdict()	DebateVerdict	Allocation engine.
puppy/quant/risk_rules.py	apply_risk_constraints()	risk flags	Allocation engine.
puppy/portfolio_ext/allocation_engine.py	decide_allocation()	AllocationDecision	Execution policy và Master Portfolio.
puppy/portfolio_ext/execution_policy.py	generate_orders()	order list	Master Portfolio.
puppy/portfolio_ext/master_portfolio.py	apply_orders(), snapshot()	PortfolioSnapshot	Evaluation và ResultStore.
puppy/evaluation/metrics.py	compute_metrics()	metrics	Backtest report.
puppy/evaluation/backtest_report.py	build_report()	report table	Paper/demo result.
puppy/evaluation/plot_metrics.py	plot_equity_curve(), plot_drawdown()	figures	Paper figures and dashboard.

7.4 Luồng xử lý một ngày giao dịch

Tại mỗi ngày giao dịch t, hệ thống chạy theo thứ tự sau:

Bước	Hàm/file chạy	Artifact sinh ra
1	MultiAssetEnvironment.step(date) trong puppy/data_engine/multi_asset_env.py	MarketStep_t
2	SectorAgent.step(MarketStep_t) trong puppy/swarm/sector_agent.py	AgentOutput_t
3	build_agent_context() trong puppy/swarm/context_builder.py	AgentContext_t
4	encode(context) trong puppy/fusion/text_encoder.py	h_text_t
5	decision_to_action_signal() trong puppy/quant/signal_utils.py; compute_log_return() trong puppy/quant/return_utils.py	QuantFeatureTable_t
6	update_shadow_portfolio() trong puppy/portfolio_ext/shadow_portfolio.py	ShadowPortfolioState_t
7	build_graph_state() trong puppy/graph/graph_builder.py; forward() trong puppy/graph/gat_model.py	GraphOutput_t
8	forward(H_text,H_graph) trong puppy/fusion/gated_fusion.py	FusionOutput_t

9	`apply_risk_constraints()` trong `puppy/quant/risk_rules.py`; `detect_conflict()` trong `puppy/debate/conflict_detector.py`	`RiskFlags_t` hoặc `DebateRequest_t`
10	`run_debate()` trong `puppy/debate/debate_loop.py`; `make_verdict()` trong `puppy/debate/judge.py` nếu cần	`DebateVerdict_t`
11	`decide_allocation()` trong `puppy/portfolio_ext/allocation_engine.py`	`AllocationDecision_t`
12	`apply_orders()` trong `puppy/portfolio_ext/master_portfolio.py`	`PortfolioSnapshot_t`

Trong đó, `ShadowPortfolioState_t` chỉ được dùng cho các vòng giao dịch sau. Không được dùng kết quả tại $t+1$ để quay lại quyết định tại ngày t, nhằm tránh data leakage.

8 Lộ trình triển khai đồng bộ theo milestone

8.1 Milestone 0 – Contract và skeleton folder

Mục tiêu: Tạo toàn bộ skeleton folder, schema chung và contract trước khi code logic sâu.

Thành viên	Việc cần làm	File/folder liên quan	Ai hỗ trợ
Đạt	Tạo skeleton cho data engine, memory store, event store.	`data_engine/`, `memory_ext/`, `common/schemas.py`	Hiệp review field liên quan return; Tiến review field liên quan memory.
Tiến	Định nghĩa schema cho AgentContext, AgentOutput, DebateRequest, DebateVerdict.	`swarm/protocols.py`, `debate/`, `common/schemas.py`	Đạt review khả năng lưu trữ; Hiệp review field dùng cho GNN/Fusion.
Hiệp	Định nghĩa schema cho QuantFeatureTable, GraphOutput, FusionOutput, AllocationDecision.	`quant/`, `graph/`, `fusion/`, `portfolio_ext/`	Đạt review EventStore; Tiến review field debate và memory sharing.
Cả nhóm	Viết tài liệu contract và luồng dữ liệu.	`docs/system_contract.md`, `docs/data_flow.md`	Cả nhóm cùng ký duyệt.

Definition of Done:

$$MarketStep \rightarrow AgentOutput \rightarrow GraphOutput \rightarrow FusionOutput \rightarrow AllocationDecision$$

phải có schema rõ ràng, owner rõ ràng và ví dụ JSON minh họa.

8.2 Milestone 1 – Vertical slice mock end-to-end

Mục tiêu: Chạy được một pipeline giả lập từ dữ liệu đến portfolio mà chưa cần LLM thật và chưa cần GAT thật.

Thành viên	Việc cần làm	File/folder liên quan	Ai hỗ trợ
Đạt	Tạo dữ liệu giả cho 3 symbol trong 5–10 ngày và trả về MarketStep.	`data_engine/multi_asset_env.py`, `scripts/run_milestone_1_mock.py`	Hiệp hỗ trợ yêu cầu price/return; Tiến hỗ trợ yêu cầu news/filings.

Tiến	Tạo FakeSectorAgent sinh decision, reason và h_text giả.	`swarm/sector_agent.py`, `swarm/protocols.py`	Hiệp hỗ trợ shape của h_text; Đạt hỗ trợ lưu AgentOutput.
Hiệp	Tính action signal, return, label, shadow return và allocation đơn giản.	`quant/signal_utils.py`, `quant/return_utils.py`, `portfolio_ext/allocation_engine.py`	Đạt hỗ trợ price matrix; Tiến hỗ trợ decision format.
Đạt	Lưu toàn bộ artifact vào EventStore.	`data_engine/event_store.py`, `data_engine/result_store.py`	Cả nhóm cung cấp schema test.

Luồng sau khi hoàn thành:

$$MarketStep_t \rightarrow FakeAgentOutput_t \rightarrow QuantFeatureTable_t \rightarrow AllocationDecision_t \rightarrow PortfolioSnapshot_t$$

8.3 Milestone 2 – Data engine và memory infrastructure thật

Mục tiêu: Thay dữ liệu giả và memory giả bằng hạ tầng thật cho swarm.

Thành viên	Việc cần làm	File/folder liên quan	Ai hỗ trợ
Đạt	Xây DuckDB/Parquet store cho price, news, filings.	`data_engine/parquet_loader.py`, `data_engine/duckdb_store.py`, `scripts/build_duckdb.py`	Hiệp hỗ trợ yêu cầu return window; Tiến hỗ trợ yêu cầu text/news.
Đạt	Tạo FAISS VectorStore và SQLite metadata store.	`memory_ext/vector_store.py`, `memory_ext/faiss_store.py`, `memory_ext/sqlite_memory_store.py`	Tiến review API retrieve memory.
Tiến	Thử gọi search_memory từ SectorAgent context builder.	`swarm/context_builder.py`	Đạt hỗ trợ API; Hiệp chưa phụ thuộc trực tiếp.
Hiệp	Kiểm tra dữ liệu giá không bị leakage và đủ để tính return/label.	`quant/return_utils.py`, `data_engine/schemas.py`	Đạt cung cấp data query.

Luồng sau khi hoàn thành:

$$DuckDB/Parquet \rightarrow MultiAssetEnvironment \rightarrow MarketStep_t \rightarrow SectorAgent \rightarrow MemoryStore$$

8.4 Milestone 3 – SectorAgent thật và Text Stream

Mục tiêu: Bọc LLMAgent gốc thành SectorAgent, tạo context đầy đủ và sinh h_text bằng pretrained encoder.

Thành viên	Việc cần làm	File/folder liên quan	Ai hỗ trợ
Tiến	Tạo SectorAgent chứa LLMAgent gốc và export AgentOutput.	`swarm/sector_agent.py`, `swarm/protocols.py`	Đạt hỗ trợ MarketStep/Memory API; Hiệp review action signal.
Tiến	Xây ContextBuilder gồm memory, shared context, shadow state, psychology state.	`swarm/context_builder.py`	Đạt hỗ trợ memory metadata; Hiệp hỗ trợ shadow state fields.
Tiến	Dùng pretrained text encoder tạo h_text.	`fusion/text_encoder.py`	Hiệp chốt dimension/format của h_text.
Đạt	Lưu AgentOutput và TextFeature vào EventStore.	`data_engine/event_store.py`	Tiến cung cấp schema; Hiệp kiểm tra đọc lại được.

Hiệp	Tạo text score head nhận h_text làm input.	`fusion/text_score_head.py`	Tiến cung cấp h_text ổn định; Đạt cung cấp dataset lưu sẵn.

Luồng sau khi hoàn thành:

$$MarketStep_t \rightarrow SectorAgent_i \rightarrow Context_{i,t} \rightarrow TextEncoder \rightarrow h_{i,t}^{text} \rightarrow AgentOutput_t$$

8.5 Milestone 4 – Shadow Portfolio và psychology feedback

Mục tiêu: Hoàn thiện vòng phản hồi từ Shadow Portfolio về SectorAgent.

Thành viên	Việc cần làm	File/folder liên quan	Ai hỗ trợ
Hiệp	Tính shadow return, ROI, rolling Sharpe, drawdown và win rate.	`portfolio_ext/shadow_portfolio.py`, `quant/shadow_metrics.py`	Đạt hỗ trợ lưu/load price và AgentOutput.
Đạt	Lưu ShadowPortfolioState theo date/symbol/agent.	`data_engine/result_store.py`	Hiệp cung cấp schema và công thức.
Tiến	Tạo PsychologyUpdate từ shadow metrics.	`swarm/psychology.py`	Hiệp tư vấn ngưỡng ROI/drawdown; Đạt cung cấp state từ store.
Tiến	Đưa psychology state vào context vòng sau.	`swarm/context_builder.py`	Đạt hỗ trợ load state; Hiệp xác nhận không leakage.

Luồng sau khi hoàn thành:

$$AgentOutput_t \rightarrow R_{t+1}^S \rightarrow ShadowPortfolioState_{t+1} \rightarrow PsychologyState_{t+1} \rightarrow Context_{t+2}$$

8.6 Milestone 5 – Graph MVP và memory sharing

Mục tiêu: Tạo graph quan hệ giữa các agent và dùng graph làm router cho memory sharing.

Thành viên	Việc cần làm	File/folder liên quan	Ai hỗ trợ
Hiệp	Tạo correlation graph, adjacency matrix và graph features.	`graph/correlation_graph.py`, `graph/graph_builder.py`, `graph/graph_features.py`	Đạt cung cấp return data; Tiến cung cấp action/h_text.
Hiệp	Tạo risk router sinh memory share routes.	`graph/risk_router.py`	Tiến review route có đủ context để inject không.
Tiến	Nhận route, lấy memory nguồn và inject sang target agent.	`memory_ext/memory_sharing.py`, `swarm/context_builder.py`	Đạt hỗ trợ memory API; Hiệp hỗ trợ alpha/edge weight.
Đạt	Lưu MemoryShareEvent.	`messaging/events.py`, `data_engine/event_store.py`	Tiến cung cấp event content; Hiệp cung cấp route.

Luồng sau khi hoàn thành:

$$QuantFeatureTable_t \rightarrow A_t, H_t^0 \rightarrow GraphOutput_t \rightarrow MemoryShareRoute_t \rightarrow SharedContext_{t+1}$$

8.7 Milestone 6 – GAT và Gated Fusion offline

Mục tiêu: Train GAT, text score head và gated fusion bằng dữ liệu đã được lưu trong EventStore.

Thành viên	Việc cần làm	File/folder liên quan	Ai hỗ trợ
Đạt	Export dataset gồm AgentOutput, h_text, returns, shadow metrics, graph data.	`data_engine/result_store.py`, `scripts/run_milestone_5_fusion.py`	Hiệp yêu cầu format; Tiến đảm bảo h_text ổn định.
Hiệp	Train text score head.	`fusion/text_score_head.py`	Tiến cung cấp h_text; Đạt cung cấp dataset.
Hiệp	Train GAT sinh h_graph và alpha matrix.	`graph/gat_model.py`, `graph/graph_dataset.py`, `graph/graph_train_loop.py`	Đạt cung cấp graph dataset.
Hiệp	Train gated fusion sinh fusion score và conflict flag.	`fusion/gated_fusion.py`, `fusion/train_loop.py`, `fusion/model_io.py`	Tiến review logic conflict phục vụ debate; Đạt lưu model artifact.
Đạt	Lưu model checkpoints và feature schema version.	`data/model_artifacts/`	Hiệp cung cấp model save/load.

Luồng sau khi hoàn thành:

$$H_t^{text}, H_t^0, A_t \rightarrow H_t^{graph}, \mathcal{A}_t \rightarrow H_t^{final} \rightarrow s_t, C_t \rightarrow FusionOutput_t$$

8.8 Milestone 7 – Allocation, Master Portfolio và Debate

Mục tiêu: Kết nối FusionOutput với AllocationDecision, Debate và Master Portfolio.

Thành viên	Việc cần làm	File/folder liên quan	Ai hỗ trợ
Hiệp	Xây risk rules và allocation engine.	`quant/risk_rules.py`, `portfolio_ext/allocation_engine.py`, `portfolio_ext/execution_policy.py`	Đạt hỗ trợ portfolio state; Tiến hỗ trợ debate verdict.
Hiệp	Phát DebateRequest khi fusion score vùng xám hoặc conflict.	`debate/conflict_detector.py`	Tiến review nội dung request.
Tiến	Xây debate loop, sector justification và judge.	`debate/prompts.py`, `debate/debate_loop.py`, `debate/judge.py`	Hiệp cung cấp risk flags; Đạt cung cấp memory/reason logs.
Hiệp	Dùng DebateVerdict để cập nhật AllocationDecision.	`portfolio_ext/allocation_engine.py`	Tiến cung cấp verdict có cấu trúc.
Đạt	Cập nhật Master Portfolio và lưu snapshot.	`portfolio_ext/master_portfolio.py`, `data_engine/result_store.py`	Hiệp cung cấp accounting logic.

Luồng sau khi hoàn thành:

$$FusionOutput_t \rightarrow RiskRules \rightarrow DebateRequest_t \rightarrow DebateVerdict_t \rightarrow AllocationDecision_t \rightarrow MasterPortfolioSnapshot_t$$

8.9 Milestone 8 – Benchmark, ablation, API và báo cáo

Mục tiêu: Đánh giá toàn bộ hệ thống và chuẩn bị kết quả phục vụ paper/demo.

Thành viên	Việc cần làm	File/folder liên quan	Ai hỗ trợ
Hiệp	Chạy benchmark và ablation.	`scripts/run_ablation.py`, `evaluation/ablation_runner.py`, `evaluation/metrics.py`	Đạt hỗ trợ batch runner; Tiến hỗ trợ variant debate/memory sharing.
Hiệp	Vẽ equity curve, drawdown curve, alpha heatmap, gate plot.	`evaluation/plot_metrics.py`, `evaluation/backtest_report.py`	Đạt cung cấp result store.
Tiến	Chọn case study memory sharing và debate.	`evaluation/explainability_report.py`, `docs/debate_examples.md`, `docs/memory_sharing_examples.md`	Đạt cung cấp logs; Hiệp cung conflict/risk context.
Đạt	Làm API và Docker để xem kết quả.	`api/`, `docker-compose.yml`, `Dockerfile`	Hiệp cung cấp metrics endpoint; Tiến cung cấp debate/memory endpoint.

Luồng sau khi hoàn thành:

$$PortfolioSnapshot \rightarrow Metrics \rightarrow Plots \rightarrow AblationReport \rightarrow API/Dashboard \rightarrow PaperFigures$$

9 Ma trận hỗ trợ giữa các thành viên

Điểm giao nhau	Owner chính	Người hỗ trợ	Quy tắc phối hợp
Data schema	Đạt	Hiệp, Tiến	Đạt định nghĩa schema; Hiệp kiểm tra đủ field tính return/label; Tiến kiểm tra đủ field cho context.
AgentOutput	Tiến	Hiệp, Đạt	Tiến xuất decision, reason, memory ids, h_text; Hiệp yêu cầu action signal chuẩn; Đạt đảm bảo lưu/load không mất field.
h_text	Tiến	Hiệp	Tiến dùng pretrained encoder tạo vector; Hiệp chốt dimension, dtype và cách batch thành H^{text}.
Shadow Portfolio	Hiệp	Đạt, Tiến	Hiệp tính ROI/drawdown/win rate; Đạt lưu state; Tiến đưa state vào psychology context.
Memory Sharing	Hiệp trigger, Tiến xử lý, Đạt lưu	Cả nhóm	Hiệp sinh route; Tiến inject context; Đạt lưu MemoryShareEvent và metadata.
Debate	Tiến	Hiệp, Đạt	Hiệp phát DebateRequest; Tiến sinh question/answer/verdict; Đạt lưu log.
Allocation	Hiệp	Tiến, Đạt	Hiệp ra target weight; Tiến cung cấp debate verdict; Đạt cập nhật Master Portfolio snapshot.
Benchmark	Hiệp	Đạt, Tiến	Hiệp tính metrics; Đạt chạy batch/export; Tiến viết case study explanation.

10 Bảng tổng hợp lộ trình file/folder theo thành viên

Thành viên	Folder chính	File quan trọng	Milestone
Đạt	`data_engine/`	`schemas.py`, `parquet_loader.py`, `duckdb_store.py`, `multi_asset_env.py`, `event_store.py`, `result_store.py`	M0–M2.
Đạt	`memory_ext/`	`vector_store.py`, `faiss_store.py`, `sqlite_memory_store.py`, `memory_namespace.py`	M2.
Đạt	`messaging/`, `api/`	`events.py`, `broker.py`, `redis_bus.py`, `main.py`, file định tuyến (routes)	M5–M8.
Tiến	`swarm/`	`protocols.py`, `sector_agent.py`, `context_builder.py`, `psychology.py`, `allocator_agent.py`	M0–M4.

Tiến	`fusion/`	`text_encoder.py`	M3–M6.
Tiến	`debate/`	`prompts.py`, `conflict_detector.py`, `debate_loop.py`, `judge.py`	M7–M8.
Hiệp	`quant/`	`signal_utils.py`, `return_utils.py`, `risk_rules.py`, `shadow_metrics.py`, `agent_reliability.py`	M1–M7.
Hiệp	`graph/`	`graph_builder.py`, `correlation_graph.py`, `graph_features.py`, `risk_router.py`, `gat_model.py`, `graph_dataset.py`, `graph_train_loop.py`	M5–M6.
Hiệp	`fusion/`	`text_score_head.py`, `gated_fusion.py`, `dataset.py`, `loss.py`, `train_loop.py`, `model_io.py`	M6.
Hiệp	`portfolio_ext/`	`shadow_portfolio.py`, `master_portfolio.py`, `execution_policy.py`, `allocation_engine.py`	M4–M7.
Hiệp	`evaluation/`	`metrics.py`, `backtest_report.py`, `plot_metrics.py`, `ablation_runner.py`	M8.

11 Definition of Done chung cho toàn dự án

Một milestone chỉ được xem là hoàn thành khi thỏa mãn đủ bốn điều kiện:

1. Có file/folder đúng như kế hoạch.
2. Có schema input/output rõ ràng.
3. Có ít nhất một script integration chạy qua artifact của milestone đó.
4. Artifact tạo ra bởi người này phải được ít nhất một người khác đọc và sử dụng thành công.

Ví dụ, `AgentOutput` chỉ được xem là hoàn thành khi:

1. Tiến tạo được `AgentOutput`.
2. Đạt lưu/load được `AgentOutput`.
3. Hiệp đọc được `AgentOutput` để tính action signal và graph feature.

12 Thứ tự ưu tiên triển khai

1. Ưu tiên 1: Contract, schema và vertical slice mock.
2. Ưu tiên 2: MultiAssetEnvironment, EventStore và SectorAgent wrapper.
3. Ưu tiên 3: Shadow Portfolio feedback và h_text.
4. Ưu tiên 4: Graph MVP và memory sharing route.
5. Ưu tiên 5: GAT, text score head và gated fusion.
6. Ưu tiên 6: Allocation engine, Master Portfolio và debate.
7. Ưu tiên 7: Benchmark, ablation, API và report.

13 Kết luận phần triển khai

Kế hoạch triển khai FINMEM Swarm phải được hiểu như một pipeline đồng bộ, trong đó ba thành viên không làm ba phần tách rời mà nối với nhau thông qua artifact chung.

$$\text{Đạt} \rightarrow \text{MarketStep, MemoryStore, EventStore} \rightarrow \text{Tiến} \rightarrow \text{AgentOutput, h}^{text} \rightarrow \text{Hiệp} \rightarrow \text{GraphOutput, FusionOutput, AllocationDecision} \rightarrow \text{Đạt} \rightarrow \text{PortfolioSnapshot, API, Report}$$

Vai trò cuối cùng của từng thành viên được chốt như sau:

- Đạt đảm bảo hệ thống có dữ liệu, memory, event store, runtime và khả năng quan sát.
- Tiến đảm bảo mỗi SectorAgent có reasoning, context, text embedding, memory sharing và debate.
- Hiệp đảm bảo tín hiệu từ agent được chuyển thành mô hình định lượng, graph, fusion, allocation và benchmark.

Nếu artifact contract được giữ ổn định, cả ba thành viên có thể phát triển song song nhưng vẫn tích hợp thành một hệ thống hoàn chỉnh.